

Análise do Impacto de Runtimes Task-Based no Desempenho de Algoritmos de Álgebra Linear Densa

Matheus Augusto Tregnago, Philippe O. A. Navaux, Lucas Mello Schnorr

¹ Institute of Informatics, Federal University of Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

{matregnago, navaux, schnorr}@inf.ufrgs.br

Resumo. *Runtimes de escalonamento de tarefas dependentes como StarPU e PaRSEC abstraem a complexidade de plataformas heterogêneas, mas adotam estratégias diferentes de escalonamento e gerenciamento de dados. Este trabalho compara o desempenho dos dois runtimes nas fatorações de Cholesky e LU sem pivotamento da biblioteca Chameleon, mantendo fixa a biblioteca de álgebra linear para isolar o efeito do runtime. Este estudo exigiu estender a interface de inserção dinâmica de tarefas do PaRSEC (DTD) e os codelets do Chameleon com suporte a GPU. Os experimentos mostram que nenhum runtime é sistematicamente superior: em máquina com GPU fraca em precisão dupla, o `parsec:gd` vence por tratar a GPU como recurso acessório; já em máquina com GPU forte, o StarPU vence com folga por ocupá-la de forma mais eficiente.*

1. Introdução

Nos últimos anos, as arquiteturas de computação de alto desempenho têm se tornado cada vez mais heterogêneas, combinando processadores multicore convencionais com aceleradores especializados, como GPUs, FPGAs e outros aceleradores. Essa diversidade de tecnologias, somada às diferenças entre os modelos de execução e as hierarquias de memória, torna difícil programar essas plataformas de forma eficiente. Uma das soluções para esse problema é a utilização de runtimes que utilizam o modelo Sequential Task Flow (STF). Nesse modelo, uma aplicação é decomposta em tarefas com dependências de dados explícitas, o que permite escalonar sua execução de forma dinâmica, conforme os recursos disponíveis [Pei et al. 2022].

Para facilitar o uso desse paradigma, diversos runtimes de escalonamento de tarefas foram criados para mapear as tarefas de uma aplicação para os recursos computacionais disponíveis, de maneira transparente ao desenvolvedor. Esses sistemas decidem quando e onde cada tarefa será executada, gerenciam as transferências de dados necessárias entre as diferentes unidades de processamento e memórias e equilibram a carga de trabalho entre os recursos [Aguerreberre 2018].

Dentre os runtimes de escalonamento de tarefas existentes, este trabalho compara especificamente o StarPU e o PaRSEC por serem dois runtimes utilizados pelo Chameleon como backend para execução das tarefas e por estarem, entre os runtimes suportados, entre os mais maduros e amplamente utilizados para algoritmos de álgebra linear densa em plataformas heterogêneas.

StarPU [Augonnet et al. 2011] é um desses runtimes. Nele, o desenvolvedor pode fornecer diferentes implementações para uma mesma tarefa, uma para CPU e outra para

GPU, por exemplo, e deixar que o próprio ambiente de execução selecione automaticamente a implementação mais adequada, considerando o ambiente computacional e a disponibilidade de recursos, buscando otimizar o desempenho. Isso isola o desenvolvedor dos detalhes da arquitetura de destino, permitindo criar aplicações mais simples e fáceis de escalar.

PaRSEC [Bosilca et al. 2011] é outro desses runtimes, com foco em ambientes de memória distribuída compostos por múltiplos nós multicore, possivelmente equipados com aceleradores. Diferentemente de runtimes baseados no modelo STF, como o StarPU, que instanciam explicitamente um nó de DAG na memória para cada tarefa submetida, o PaRSEC representa as dependências entre tarefas de forma compacta e simbólica, o que lhe permite escalar para milhões de tarefas com baixo overhead de memória. O escalonamento é totalmente distribuído – cada nó executa sua própria instância do escalonador, sem coordenação central – o que favorece a sobreposição entre computação e comunicação.

Embora StarPU e PaRSEC compartilhem o objetivo de abstrair o escalonamento de tarefas em plataformas heterogêneas, os escalonadores de cada runtime avaliados neste trabalho diferem numa dimensão central: o StarPU decide o uso da GPU dinamicamente, orientado por um modelo de desempenho aprendido a partir do histórico de execuções, enquanto os escalonadores do PaRSEC decidem de forma estática, sem medir tempos reais de execução. Essa diferença de filosofia sugere, à primeira vista, que o StarPU deveria ter vantagem sistemática sobre o PaRSEC em algoritmos de álgebra linear densa; um dos objetivos deste trabalho é verificar empiricamente se essa expectativa se confirma.

Diversos trabalhos avaliam runtimes task-based aplicados a algoritmos de álgebra linear densa, mas cada um deles isola apenas parte do problema, limitando a comparação a um único runtime ou comparando runtimes distintos por meio de bibliotecas de álgebra linear diferentes.

Hoque et al. [Hoque et al. 2017] comparam os dois paradigmas de programação oferecidos pelo PaRSEC: o Parameterized Task Graph (PTG) e o Dynamic Task Discovery (DTD) utilizando o Chameleon em conjunto com PaRSEC, StarPU e QUARK. A avaliação, no entanto, é restrita a execuções somente em CPU, sem considerar aceleradores.

Lisito et al. [Lisito et al. 2025] avaliam a fatoração LU com pivotamento parcial, comparando o StarPU (com o Chameleon) e o PaRSEC (com o DPLASMA). Como cada runtime é avaliado em conjunto com uma biblioteca de álgebra linear diferente, eventuais diferenças de desempenho observadas podem ser atribuídas tanto ao runtime quanto à implementação específica de cada biblioteca.

Agullo et al. [Agullo et al. 2015] comparam o desempenho da fatoração de Cholesky sobre o StarPU com limites teóricos de desempenho em plataformas heterogêneas. O trabalho, contudo, considera apenas o StarPU, sem comparação direta com outro runtime de escalonamento de tarefas.

Não foi encontrado nenhum trabalho anterior que compara StarPU e PaRSEC utilizando a mesma biblioteca de álgebra linear densa em um ambiente heterogêneo. Esse é o diferencial deste trabalho, no qual foi fixado o Chameleon como biblioteca única e variado apenas o runtime e o escalonador utilizado.

Este trabalho utiliza o Chameleon [Agullo et al. 2012], uma biblioteca de álgebra linear densa que implementa fatorações como LU, Cholesky e QR por meio de algoritmos por blocos [Buttari et al. 2007], delegando o escalonamento de suas tarefas a runtimes externos, o StarPU e o PaRSEC. Os experimentos foram executados em duas partições do cluster PCAD, no INF/UFRGS: (i) tupi, equipada com um processador Intel Core i9-14900KF (24 núcleos físicos, 3,2 GHz) e uma GPU NVIDIA GeForce RTX 4090 (24 GB); (ii) poti, equipada com um processador Intel Core i7-14700KF (20 núcleos físicos, 3,4 GHz) e uma GPU NVIDIA GeForce RTX 4070 (12 GB).

O restante deste trabalho está organizado da seguinte forma. A Seção 2 descreve os conceitos utilizados neste trabalho, enquanto a Seção 3 apresenta os métodos e materiais utilizados. A Seção 4 detalha nossa metodologia experimental, seguida pelos resultados experimentais na Seção 5. Por fim, a Seção 6 conclui este trabalho.

2. Fundamentação Teórica

Nesta seção, apresentamos uma visão geral dos runtimes de escalonamento de tarefas, StarPU e PaRSEC, e da biblioteca de álgebra linear densa Chameleon utilizados neste trabalho.

2.1. StarPU

StarPU [Augonnet et al. 2011] é um framework escrito em C para programação orientada a tarefas em plataformas híbridas compostas por CPUs e GPUs, com suporte também a ambientes distribuídos por meio da extensão StarPU-MPI. As tarefas de uma aplicação StarPU são definidas por meio de codelets, estruturas que agrupam uma ou mais implementações de uma mesma operação, além da forma como cada um de seus parâmetros é acessado. A partir da ordem de submissão das tarefas e dos modos de acesso declarados para cada dado, o StarPU desenrola implicitamente, tarefa a tarefa, o grafo acíclico dirigido (DAG) completo de dependências, sem que o programador precise expressá-lo manualmente. Em tempo de execução, o runtime escolhe qual das implementações disponíveis será utilizada para cada tarefa, considerando a disponibilidade dos recursos, a distribuição de carga entre os workers e o custo das cópias de dados envolvidas. O gerenciamento de memória nos aceleradores também é responsabilidade do StarPU, que realiza alocação e liberação de buffers na GPU, cópias entre a memória do host e do dispositivo, e a sobreposição dessas transferências com a computação por meio de prefetching.

Neste trabalho utilizamos dois escalonadores do StarPU: o Deque Model Data Aware (dmda), que mantém um modelo de desempenho por par tarefa-recurso, estimado a partir do histórico de execuções, e atribui cada tarefa ao recurso que minimiza o tempo estimado de conclusão, somando o tempo de computação previsto ao custo estimado de transferência de dados; e o dmdas, uma variante que processa as tarefas seguindo a ordem de prioridade dada pela distância ao caminho crítico do DAG, ao custo de uma ordenação adicional.

2.2. PaRSEC

PaRSEC [Bosilca et al. 2011] é um framework para escalonamento de tarefas voltado a ambientes de memória distribuída compostos por múltiplos nós multicore. Diferentemente de runtimes STF como o StarPU, que desenrolam explicitamente, tarefa a tarefa,

um DAG completo de dependências, o PaRSEC representa o grafo de dependências de forma compacta e simbólica, por meio de um formato conhecido como Parameterized Task Graph (PTG), também chamado de Job Data Flow (JDF). O escalonamento é totalmente distribuído, cada nó executa sua própria instância do escalonador, que utiliza filas locais por thread e mecanismos de roubo de tarefas sensíveis à hierarquia NUMA para favorecer o reaproveitamento de cache. Assim como o StarPU sobrepõe transferências e computação por meio de prefetching, o PaRSEC sobrepõe computação e comunicação entre nós: as comunicações são implícitas, inferidas diretamente das dependências de dados do grafo, e tratadas de forma assíncrona por uma thread dedicada. Assim como o StarPU, o PaRSEC também oferece suporte a aceleradores por meio de múltiplas implementações para uma mesma tarefa, permitindo que o escalonador decida em tempo de execução em qual unidade.

Neste trabalho utilizamos dois escalonadores do PaRSEC: o Global Dequeue (`gd`), que mantém uma única fila dupla compartilhada por todos os workers, com balanceamento natural de carga mas sem localidade de dados nem roubo de tarefas; e o Local Flat Queues (`lfq`), em que cada worker mantém uma fila local ordenada por prioridade e recorre ao roubo de tarefas das filas vizinhas, na ordem de distância na hierarquia de hardware, quando fica ocioso.

2.3. Chameleon

Chameleon [Agullo et al. 2012] é uma biblioteca escrita em C que fornece rotinas para a resolução de sistemas lineares gerais densos, sistemas simétricos positivos definidos e problemas de mínimos quadrados lineares, por meio das fatorações LU, Cholesky, QR e LQ. Assim como as bibliotecas PLASMA e MAGMA, o Chameleon segue a abordagem de algoritmos por blocos [Buttari et al. 2007], em que cada fatoração é decomposta em uma sequência de tarefas de granularidade fina que operam sobre submatrizes quadradas, reduzindo a necessidade de sincronizações globais e expondo um grau de paralelismo bem maior do que o obtido pelas versões tradicionais orientadas a blocos de linha/coluna, como as do LAPACK e do ScaLAPACK. A fatoração de Cholesky por blocos, por exemplo, é expressa como uma sequência de chamadas às rotinas POTRF, TRSM, SYRK e GEMM, cada uma correspondendo a uma tarefa cujas dependências de dados definem um DAG. Em vez de escalonar essas tarefas de forma estática, o Chameleon delega essa responsabilidade a um runtime de escalonamento de tarefas, como o StarPU, o PaRSEC ou o OpenMP, enquanto a implementação interna de cada tarefa reutiliza rotinas clássicas de alto desempenho. Essa separação entre a descrição do algoritmo, em termos de tarefas e dependências, e sua execução propriamente dita é o que permite ao Chameleon explorar arquiteturas heterogêneas compostas por CPUs e GPUs sem que o algoritmo precise ser reescrito para cada plataforma.

3. Métodos e Materiais

Ao longo deste trabalho, identificamos que a comparação entre StarPU e PaRSEC não era diretamente possível a partir das versões atuais disponíveis do Chameleon e do PaRSEC, pois, enquanto os codelets do Chameleon para o StarPU já possuem suporte maduro a GPU, a interface de inserção dinâmica de tarefas do PaRSEC (DTD) não suportava a execução de tarefas em GPU, e os codelets do Chameleon para o PaRSEC estavam limitados

à CPU. Esta seção descreve as principais modificações que desenvolvemos no PaRSEC e no Chameleon para permitir essa comparação.

A interface DTD do PaRSEC permitia associar a cada tarefa apenas uma implementação em CPU, diferentemente da interface estática PTG, que já suporta múltiplas implementações por tarefa. Adicionamos ao PaRSEC uma nova função, que permite registrar, além da implementação em CPU, uma implementação em GPU para a mesma tarefa. Em tempo de execução, o escalonador passa a tentar primeiro a implementação em GPU, delegando a submissão ao mesmo mecanismo de escalonamento de GPU já utilizado pelo backend PTG, usando a implementação em CPU apenas quando nenhum dispositivo está disponível.

Essa mudança expôs um problema de coerência de dados: quando uma tarefa executada em CPU escrevia um bloco, o caminho de execução da DTD não invalidava corretamente eventuais cópias desse bloco já presentes na memória da GPU. Uma tarefa de GPU subsequente podia então ler dados desatualizados, produzindo resultados numericamente incorretos. Corrigimos o problema invalidando explicitamente as cópias de dispositivo de um bloco sempre que uma tarefa de CPU o modifica.

Com o suporte a GPU já disponível na interface DTD do PaRSEC, implementamos, no lado do Chameleon, as implementações em GPU dos codelets utilizados pelas fatorações avaliadas neste trabalho, que até então só possuíam implementação em CPU quando executadas com o PaRSEC. Cada uma dessas implementações delega o cálculo às bibliotecas cuBLAS e cuSOLVER, de forma análoga aos codelets equivalentes já existentes para o StarPU, associando-as à sua contraparte de CPU por meio da nova função de inserção de tarefas com suporte a CUDA.

4. Metodologia

Nesta seção, apresentamos nossa metodologia experimental. Primeiro, damos uma visão geral das aplicações de álgebra linear densa consideradas neste estudo, depois, descrevemos o ambiente de execução utilizado nos experimentos e, por fim, detalhamos o projeto experimental adotado.

4.1. Aplicações

Consideramos dois algoritmos de fatoração disponíveis no Chameleon: a fatoração de Cholesky (`potrf`) e a fatoração LU sem pivotamento (`getrf_nopiv`). A fatoração de Cholesky decompõe uma matriz real simétrica positiva definida A , de ordem $n \times n$, na forma $A = LL^T$, em que L é uma matriz real triangular inferior de ordem $n \times n$ com elementos diagonais positivos. O algoritmo por blocos utiliza quatro tipos de tarefa: POTRF, SYRK, TRSM e GEMM, todas com implementações em CPU e em GPU. Já a fatoração LU sem pivotamento decompõe uma matriz real A , de ordem $n \times n$, na forma $A = LU$, em que L é triangular inferior e U é triangular superior. O projeto desse algoritmo por blocos para sistemas acelerados por GPU é descrito por Agullo et al. [Agullo et al. 2011], e utiliza três tipos de tarefa: GETRF, TRSM e GEMM, das quais apenas GETRF, que fatora o bloco da diagonal, está restrita à CPU.

4.2. Ambiente Experimental

A Tabela 1 especifica o hardware das duas partições do cluster PCAD, no INF/UFRGS, utilizadas nos experimentos: tupi e poti.

Tabela 1. Especificação de hardware das partições utilizadas.

Partição	CPU	P-cores	RAM	GPU	GFLOPS teóricos em FP64
tupi	Intel Core i9-14900KF, 3,2 GHz, 24 núcleos	8	128 GB	NVIDIA 4090, 24 GB	1.290
poti	Intel Core i7-14700KF, 3,4 GHz, 20 núcleos	8	96 GB	NVIDIA 4070, 12 GB	455

A coluna P-cores indica o número de núcleos de alto desempenho; os núcleos restantes até o total informado na coluna CPU são núcleos de eficiência. A última coluna traz o pico teórico de desempenho de cada GPU em precisão dupla.

Do ponto de vista de software, utilizamos o Chameleon na versão 1.4.0, com suporte a CUDA habilitado. Como runtimes de escalonamento, utilizamos o StarPU 1.4.7 e uma versão do PaRSEC derivada de um fork mantido por desenvolvedores do próprio Chameleon, ambos também compilados com suporte a CUDA e com a coleta de rastros de execução habilitada. Para equalizar o número de fluxos de execução em GPU entre os dois runtimes, fixamos STARPU_NWORKER_PER_CUDA=4; no PaRSEC esse número é definido em tempo de compilação (PARSEC_MAX_STREAMS=6, das quais quatro são de execução e duas de transferência), sem parâmetro equivalente configurável em tempo de execução. Utilizamos CUDA 12.4, gcc 13.3.0 e Debian (?) como sistema operacional, com driver NVIDIA (?). A reprodutibilidade de todo o ambiente é garantida por meio do gerenciador de pacotes Nix, com as dependências fixadas em um flake, publicado em <https://github.com/matreagnago/intro-hpc>. A análise dos dados é realizada após a execução por meio de scripts escritos na linguagem R, utilizando o framework StarVZ [Pinto et al. 2021] para a visualização dos rastros de execução coletados pelo StarPU e pelo PaRSEC.

4.3. Projeto Experimental

Realizamos três projetos experimentais complementares para investigar a influência do escalonador, do tamanho do problema e da máquina alvo no desempenho das fatorações de Cholesky (potrf) e LU sem pivotamento (getrf_nopiv). O primeiro projeto experimental determina o tamanho de bloco ideal para cada máquina, um parâmetro que precisa estar fixo antes de comparar os runtimes entre si. O segundo projeto experimental, um projeto fatorial completo, usa esse tamanho de bloco ótimo para comparar o desempenho do StarPU e do PaRSEC em função do tamanho do problema. O terceiro projeto experimental usa rastros de execução para explicar, em nível de tarefa, a origem das diferenças de desempenho observadas no segundo projeto experimental. Em todos eles, a variável de resposta é o GFLOPS, bilhões de operações de ponto flutuante por segundo, relatado pela própria aplicação ao final da execução.

O primeiro projeto experimental tem como objetivo determinar o tamanho de bloco ideal para cada máquina, por meio de uma varredura sistemática de configurações de bloco em precisão dupla. Fixamos o tamanho da matriz de entrada em $N = 40.000$ com precisão dupla e variamos o tamanho do bloco dentro de um conjunto de valores. Escolhemos $N = 40.000$ por ser grande o suficiente para saturar os recursos computaci-

onais e expor com clareza o efeito do tamanho de bloco sobre o desempenho. Cruzamos essas configurações com as duas fatorações (`potrf`, `getrf_nopiv`) e quatro combinações de runtime e escalonador. Os dois escalonadores do StarPU são guiados por um modelo de desempenho medido em tempo de execução, enquanto os dois do PaRSEC decidem o uso da GPU de forma estática. A expectativa que orienta todos os três projetos experimentais, portanto, é a de que `starpu:dmda` e `starpu:dmdas` superem as duas configurações do PaRSEC. Além disso, executamos o mesmo desenho experimental em ambas as partições do PCAD, `tupi` e `poti`, variando apenas o número de threads de acordo com o número total de núcleos físicos disponíveis em cada uma. A ordem de execução foi aleatória, com cinco repetições por configuração, para caracterizar a variabilidade entre execuções de uma mesma configuração. O pico de GFLOPS observado em cada máquina, em precisão dupla, determina o tamanho de bloco utilizado no segundo e no terceiro projetos experimentais.

O segundo projeto experimental, um projeto fatorial completo, investiga os efeitos de interação entre três fatores que influenciam o desempenho das fatorações em precisão dupla: a máquina utilizada (`tupi` ou `poti`), a combinação de runtime e escalonador, e o tamanho do problema, fixando o tamanho de bloco no valor ótimo identificado para cada máquina no primeiro projeto experimental. Como esse valor de bloco determina a granularidade computacional disponível para cada GPU, o intervalo de valores de N explorado também foi definido por máquina: dez níveis entre 5.000 e 60.000 na `tupi`, e seis níveis entre 5.000 e 60.000 na `poti`. Para cada uma das duas fatorações, executamos todas as combinações desse desenho fatorial em ordem aleatória, com três repetições na `tupi` e cinco na `poti`.

Por fim, o terceiro projeto experimental tem como objetivo explorar como cada escalonador se comporta em cada uma das aplicações e máquinas. Empregamos os mecanismos de coleta de rastros de execução do StarPU e do PaRSEC, por meio de oito execuções representativas por máquina, uma para cada combinação entre as duas fatorações e as quatro combinações de runtime e escalonador. Como o StarVZ foi originalmente projetado para consumir rastros do StarPU, adaptamos o pipeline de análise para o PaRSEC, onde os rastros brutos são primeiro convertidos para o formato Paje por meio da ferramenta `dbp2paje` do próprio PaRSEC, preservando os nomes de containers necessários para o StarVZ identificar corretamente workers e streams de GPU. A partir do Paje gerado, o restante do pipeline de análise é idêntico ao utilizado para o StarPU, permitindo comparar os dois runtimes em um único ambiente de análise de rastros.

5. Resultados

Esta seção apresenta os resultados dos três projetos experimentais descritos na Seção 4, na mesma ordem em que eles se encadeiam. A Seção 5.1 usa a varredura de tamanho de bloco para fixar esse parâmetro em cada máquina. A Seção 5.2 usa esse valor para comparar os quatro pares de runtime e escalonador em função do tamanho do problema. A Seção 5.3 recorre aos rastros de execução para explicar, em nível de tarefa, por que o runtime vencedor muda de uma máquina para a outra.

5.1. Escolha do tamanho de bloco

A Figura 1 mostra o resultado da varredura de tamanho de bloco para as quatro combinações de runtime e escalonador em precisão dupla, com $N = 40.000$ fixo em cada

máquina.

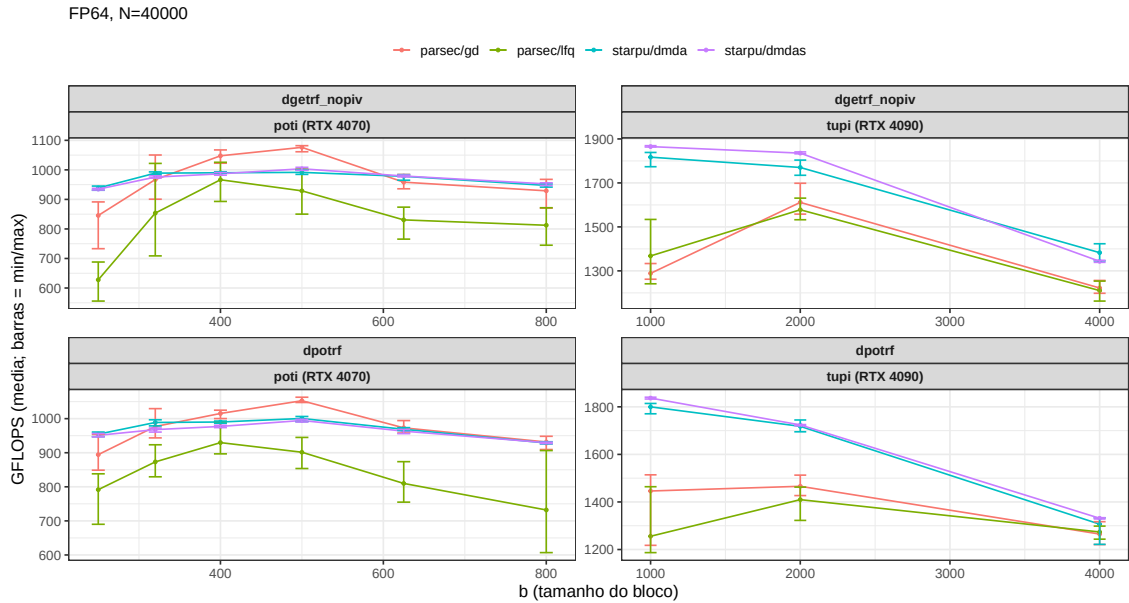


Figura 1. GFLOPS em função do tamanho de bloco b , por algoritmo e máquina.

A figura traz o GFLOPS médio das cinco repetições em função do tamanho de bloco b . Cada ponto é uma configuração medida, os pontos de uma mesma configuração são ligados por uma linha, e a barra vertical marca o mínimo e o máximo observados entre as repetições. A cor identifica o runtime, e o tom e o marcador identificam o escalonador dentro dele: o StarPU aparece em tons de vermelho (`starpu:dmda` e `starpu:dmdas`) e o PaRSEC em tons de azul (`parsec:lfq` e `parsec:gd`). Os painéis estão dispostos em uma grade: cada linha é uma das duas fatorações e cada coluna é uma das duas máquinas. Como as duas máquinas foram varridas em grades de b distintas e os patamares de GFLOPS diferem entre elas, os eixos não são compartilhados entre painéis.

Na `poti`, as quatro configurações convergem para um pico em torno de $b = 500$. Na `tupi`, por outro lado, há uma divergência estrutural entre os dois runtimes: o StarPU tem o seu pico em $b = 1.000$, enquanto o PaRSEC só atinge seu melhor resultado em $b = 2.000$ e cai mais acentuadamente para blocos menores ou maiores que o seu próprio ótimo. Os tamanhos de bloco adotados nas duas etapas seguintes são os que maximizam o GFLOPS em cada máquina: $b = 1.000$ na `tupi` e $b = 500$ na `poti`.

5.2. Desempenho em função do tamanho do problema

A Figura 2 apresenta o resultado do segundo projeto experimental.

A figura traz o GFLOPS médio em função da ordem N da matriz de entrada, com cada máquina rodando no seu tamanho de bloco ótimo. Cada ponto é a média das repetições daquela configuração naquele N , os pontos são ligados por uma linha, e a barra vertical corresponde a um desvio-padrão para cada lado. Cores e marcadores identificam as configurações como na Figura 1. Os painéis novamente formam uma grade de fatoração por máquina; aqui, porém, o eixo horizontal é comum a todos os painéis e apenas o eixo

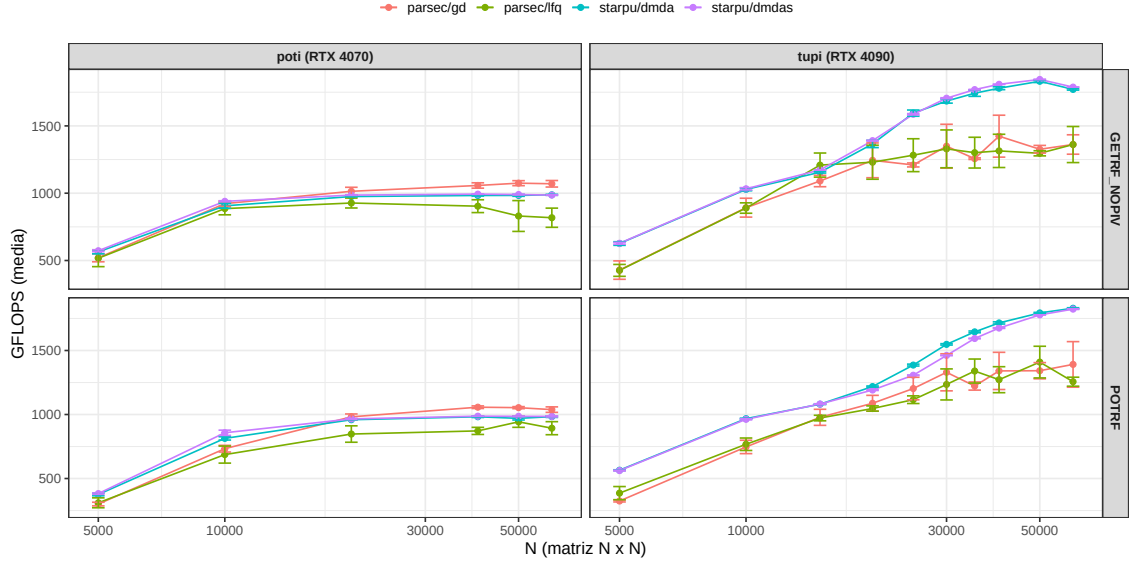


Figura 2. GFLOPS médio em função de N , por algoritmo e máquina.

vertical é livre por linha, de modo que a comparação entre as duas máquinas para uma mesma fatoração seja direta. O eixo vertical inclui o zero, para que as diferenças de altura entre as curvas possam ser lidas em proporção.

O padrão que aparece na Figura 2 é a observação central que organiza o restante desta seção. Na poti, o StarPU lidera até $N \approx 10.000$, mas a partir de $N \approx 20.000$ o `parsec:gd` assume a liderança e a mantém até $N = 60.000$, o maior valor testado. Na tupi, por sua vez, o StarPU lidera em toda a faixa de N , com a vantagem sobre o PaRSEC aumentando à medida que N cresce. Esse resultado contraria a intuição de que o StarPU, por manter um modelo de desempenho aprendido e refinado em tempo de execução, deveria vencer sistematicamente o PaRSEC, que decide o escalonamento de GPU de forma estática desconsiderando modelos de desempenho, sem medir tempos reais de execução. A próxima seção usa os rastros de execução coletados nas configurações de $N = 40.000$ para explicar por que o resultado se inverte entre as duas máquinas.

5.3. Análise de rastros de execução

Para entender a inversão observada na seção anterior, analisamos em detalhe os rastros de execução coletados para a fatoração de Cholesky com $N = 40.000$ e o bloco ideal de cada máquina: $b = 1.000$ na tupi e $b = 500$ na poti.

O ponto de partida é a relação entre a capacidade de cálculo da GPU e a da CPU em cada máquina, medida nos próprios rastros a partir da tarefa `gemm`, que responde pela quase totalidade das operações de ponto flutuante das duas fatorações. Na poti, a RTX 4070 sustenta 429 GFLOPS contra 601 a 768 GFLOPS dos 19 *workers* de CPU, conforme a configuração: a GPU vale pouco mais da metade da CPU. Na tupi, a relação se inverte, com a RTX 4090 sustentando 1.219 GFLOPS contra 698 a 740 GFLOPS dos 23 *workers*, quase o dobro da CPU.

A Figura 3 decompõe, na poti, o desempenho de cada configuração nas parcelas entregues por cada classe de recurso. O eixo vertical traz o GFLOPS entregue, obtido

dividindo as operações de ponto flutuante executadas por uma classe pelo *makespan* da execução; as duas parcelas de uma mesma configuração são empilhadas, de modo que a altura total da barra é o desempenho da execução, repetido no rótulo acima dela. Cada barra é uma das quatro configurações, a cor distingue a classe de recurso e os dois painéis separam as duas fatorações. Diferentemente das Figuras 1 e 2, aqui a cor codifica a classe de recurso, e não a configuração.

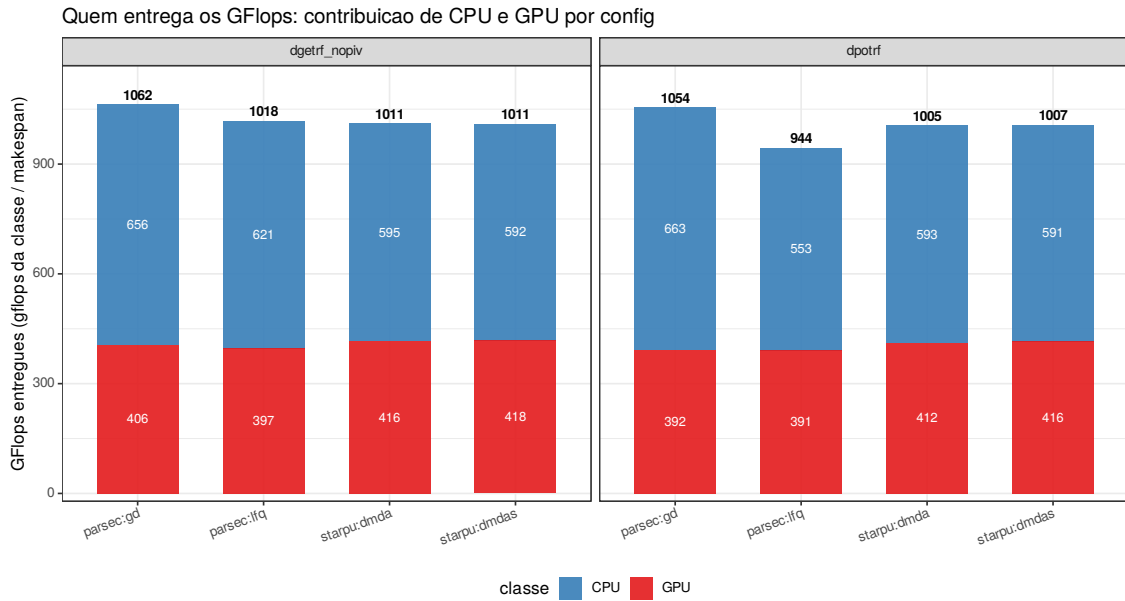


Figura 3. GFLOPS entregues por classe de recurso (CPU/GPU) na poti, por algoritmo e configuração.

A leitura da figura desfaz a explicação intuitiva para o resultado da poti. A parcela entregue pela GPU é praticamente a mesma nas quatro configurações, entre 391 e 416 GFLOPS na fatoração de Cholesky, apesar de os quatro escalonadores decidirem o uso da GPU de maneiras bastante diferentes. O que separa as configurações é a parcela entregue pela CPU, que varia de 553 a 663 GFLOPS. É na CPU, e não na GPU, que a corrida se decide na poti, e cada configuração chega ao seu resultado por um caminho distinto.

O `parsec:gd` vence porque preserva a CPU. Ele trata a GPU como recurso acessório, despachado de forma oportunista pelo próprio *worker* que obtém acesso ao dispositivo, sem nenhuma thread reservada para essa gestão; seus 19 *workers* permanecem inteiramente disponíveis para o cálculo e entregam 663 GFLOPS, a maior parcela de CPU entre as quatro configurações. O StarPU coloca 41% das operações na GPU, contra 37% do `parsec:gd`, e essa é de fato a divisão que maximiza o desempenho do sistema que o seu modelo enxerga. Para equalizar o número de fluxos de execução em GPU entre os dois runtimes, configuramos o StarPU com quatro *workers* CUDA, e essas quatro threads disputam os mesmos núcleos físicos usados pelos *workers* de CPU comuns: a tarefa *gemm* executada em CPU custa 7,81 ms no `starpu:dmda` contra 6,19 ms no `parsec:gd`, 26% a mais. Cabe registrar que essas quatro threads são uma escolha de configuração nossa, feita para tornar a comparação justa, e não uma propriedade intrínseca do StarPU.

O `parsec:lfq`, por sua vez, mostra que não basta ao PaRSEC deixar a CPU livre, é preciso ocupá-la: ele entrega a menor parcela de CPU das quatro configurações,

553 GFLOPS, não por custo de gestão da GPU, mas porque seus *workers* ficam ociosos em 27% do *makespan*, contra 13% do `parsec:gd` e cerca de 2% das duas configurações do StarPU.

5.3.1. Painéis espaço-tempo e progressão do DAG

As duas figuras seguintes são painéis do StarVZ [Pinto et al. 2021, Garcia Pinto et al. 2018], e ambas trazem o tempo de execução no eixo horizontal, compartilhado entre os quatro painéis para que os *makespans* possam ser comparados diretamente. Na Figura 4, cada linha do eixo vertical é um recurso de execução: os 19 *workers* de CPU e, abaixo deles, as *lanes* de GPU. Cada retângulo é uma tarefa, posicionada no recurso em que executou e colorida pelo seu tipo (`potrf`, `trsm`, `syrk` ou `gemm`); espaço em branco significa recurso ocioso. Para não sobrepor rótulos, apenas o primeiro *worker* de CPU do nó é identificado no eixo. Cada um dos quatro painéis é uma das configurações.

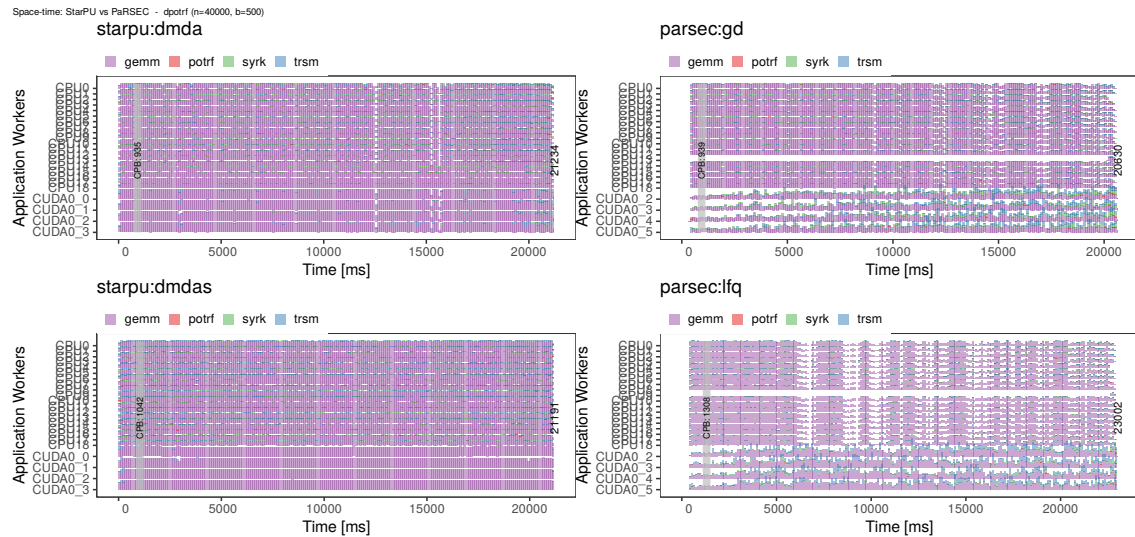


Figura 4. Painéis espaço-tempo de StarPU e PaRSEC na poti.

A figura dá forma visual à decomposição da seção anterior. O StarPU usa quatro *lanes* de GPU, uma por *worker* CUDA, ocupadas por `gemm` praticamente do início ao fim da execução. O PaRSEC dispõe de seis *streams* CUDA, mas apenas quatro delas executam tarefas; as outras duas são reservadas às transferências entre a memória do *host* e a do dispositivo (H2D, do *host* para o dispositivo, e D2H, do dispositivo para o *host*) e, por não carregarem computação, não aparecem no painel. A comparação é, portanto, de quatro *lanes* de execução contra quatro, o que foi deliberado; a diferença é que no StarPU esse número é um parâmetro de tempo de execução e no PaRSEC ele é definido em tempo de compilação. Nas faixas de CPU, por outro lado, as quatro configurações não se comportam da mesma forma: as duas do StarPU e, em menor grau, o `parsec:gd` aparecem saturadas, enquanto o `parsec:lfq` exibe lacunas brancas largas e recorrentes ao longo de toda a execução – são esses intervalos que aparecem como os 27% de ociosidade de CPU relatados na seção anterior.

A Figura 5 usa o mesmo eixo horizontal de tempo, mas traz no eixo vertical o índice k da iteração da fatoração à qual cada tarefa pertence. Cada marca é uma tarefa, colorida pelo tipo como na figura anterior, e as duas linhas pretas que envolvem a nuvem de pontos ligam, para cada iteração, a primeira e a última tarefa executadas: a distância entre elas mede o quanto o runtime mantém iterações simultaneamente em andamento. Novamente há um painel por configuração. O `starpu:dmda` progride de forma quase linear, com as duas linhas praticamente coladas, o que significa que ele conclui uma iteração antes de avançar para a seguinte. O `starpu:dmdas` abre um envelope largo, sinal de que sua política de priorização pelo caminho crítico adianta tarefas de iterações futuras. Já o `parsec:gd` e o `parsec:lfq` progridem de forma irregular, em degraus, com o `parsec:lfq` exibindo os patamares mais longos, nos mesmos instantes em que a Figura 4 mostra a CPU ociosa.

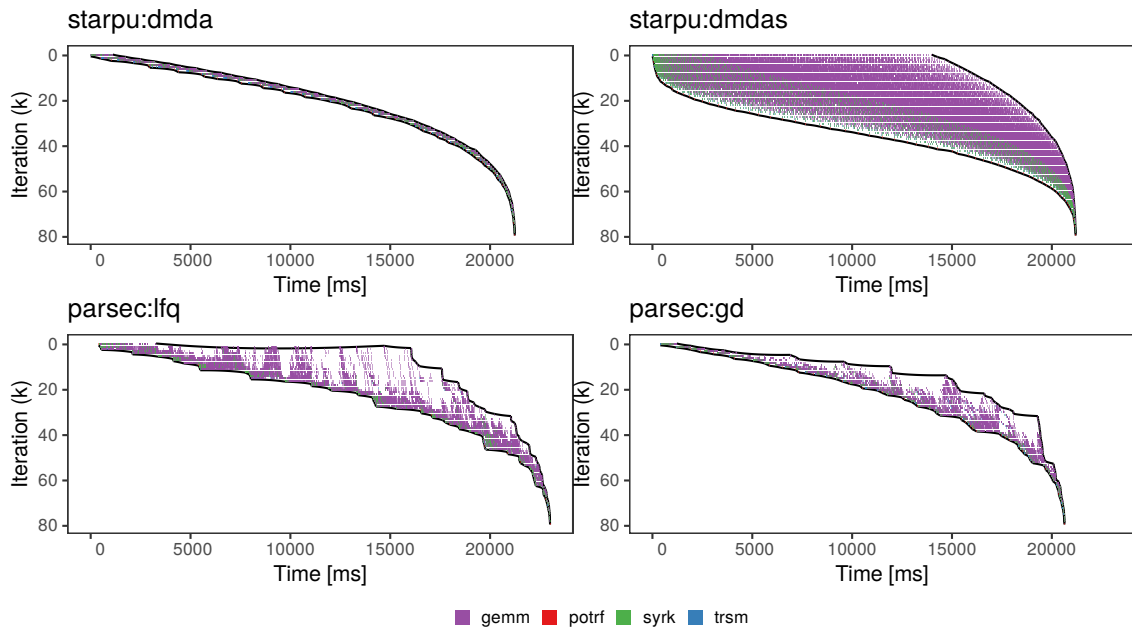


Figura 5. Progressão das iterações da fatoração de Cholesky ao longo do tempo, na poti.

6. Conclusão

Este trabalho comparou o desempenho dos runtimes StarPU e PaRSEC nas fatorações de Cholesky e LU sem pivotamento da biblioteca Chameleon, mantendo fixa a biblioteca de álgebra linear para isolar o efeito do runtime sobre o desempenho. Para viabilizar essa comparação, estendemos a interface DTD do PaRSEC com suporte à execução de tarefas em GPU e implementamos os codelets de GPU correspondentes no Chameleon, capacidades até então restritas ao caminho do StarPU.

Os resultados mostram que o runtime vencedor depende do equilíbrio entre a capacidade de CPU e de GPU de cada máquina. Na poti, onde a GPU vale pouco mais da metade da capacidade agregada da CPU em precisão dupla, o `parsec:gd` venceu por tratar a GPU como recurso acessório, sem impor custo estrutural sobre os *workers* de CPU. Na tupi, onde a GPU vale quase o dobro da CPU, o StarPU venceu com folga. A análise dos rastros de execução mostrou que a inversão não se decide pela eficiência com

que cada runtime executa uma tarefa na GPU: na poti, as quatro configurações extraem essencialmente o mesmo desempenho do dispositivo, e o que as separa é o quanto sobra de CPU. O StarPU reserva *workers* à gestão da GPU, o que encarece em 26% cada tarefa executada em CPU, e recupera esse custo apenas quando a GPU é forte o bastante para justificá-lo.

Como trabalho futuro, pretendemos: (i) instrumentar o PaRSEC para medir a duração de tarefas de GPU diretamente no dispositivo, em vez de no *host*, eliminando a sobreposição entre *streams* que hoje impede comparar durações de tarefas de GPU entre os dois runtimes; (ii) estender a avaliação a um espectro maior de relações entre capacidade de CPU e de GPU, com outras GPUs e CPUs, para caracterizar com mais precisão o limiar em que a vantagem se inverte entre os runtimes; (iii) avaliar o desempenho em ambientes distribuídos multi-nó e multi-GPU, nos quais o escalonamento totalmente distribuído do PaRSEC foi projetado para ter vantagem sobre o modelo de gerenciamento por nó do StarPU; e (iv) estender as extensões de GPU desenvolvidas na interface DTD do PaRSEC a outras faturações do Chameleon, como QR.

Acknowledgments

Who paid for this?

Referências

- Aguerrebere, C. (2018). *Statistical Methods for Computational Photography*. Habilitation à diriger des recherches (hdr), Université Paris-Saclay. Inria, HAL Id: tel-01959127v1.
- Agullo, E., Augonnet, C., Dongarra, J., Faverge, M., Langou, J., Ltaief, H., and Tomov, S. (2011). Lu factorization for accelerator-based systems. In *2011 9th IEEE/ACS International Conference on Computer Systems and Applications (AICCSA)*, pages 217–224.
- Agullo, E., Augonnet, C., Dongarra, J., Ltaief, H., Namyst, R., Thibault, S., and Tomov, S. (2012). Chapter 34 - a hybridization methodology for high-performance linear algebra software for gpus. In mei W. Hwu, W., editor, *GPU Computing Gems Jade Edition*, Applications of GPU Computing Series, pages 473–484. Morgan Kaufmann, Boston.
- Agullo, E., Beaumont, O., Eyraud-Dubois, L., Herrmann, J., Kumar, S., Marchal, L., and Thibault, S. (2015). Bridging the gap between performance and bounds of cholesky factorization on heterogeneous platforms. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*, pages 34–45.
- Augonnet, C., Thibault, S., Namyst, R., and Wacrenier, P.-A. (2011). Starpu: a unified platform for task scheduling on heterogeneous multicore architectures. *Concurrency and Computation: Practice and Experience*, 23(2):187–198.
- Bosilca, G., Bouteiller, A., Danalis, A., Herault, T., Lemariner, P., and Dongarra, J. (2011). DAGuE: a generic distributed DAG engine for high performance computing. In *Proceedings of the Workshops of the 25th IEEE International Symposium on Parallel and Distributed Processing (IPDPS 2011 Workshops)*, pages 1151–1158, Anchorage, Alaska, USA. IEEE.
- Buttari, A., Langou, J., Kurzak, J., and Dongarra, J. J. (2007). A class of parallel tiled linear algebra algorithms for multicore architectures. *CoRR*, abs/0709.1272.

- Garcia Pinto, V., Mello Schnorr, L., Stanisic, L., Legrand, A., Thibault, S., and Danjean, V. (2018). A visual performance analysis framework for task-based parallel applications running on hybrid clusters. *Concurrency and Computation: Practice and Experience*, 30(18):e4472.
- Hoque, R., Herault, T., Bosilca, G., and Dongarra, J. (2017). Dynamic task discovery in parsec: a data-flow task-based runtime. In *Proceedings of the 8th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems, Scala '17*, New York, NY, USA. Association for Computing Machinery.
- Lisito, A., Faverge, M., Kuhn, M., Pruvost, F., and Ramet, P. (2025). Scalable and portable lu factorization with partial pivoting on top of runtime systems. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1011–1022.
- Pei, Y., Bosilca, G., and Dongarra, J. (2022). Sequential task flow runtime model improvements and limitations. In *2022 IEEE/ACM International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*, pages 1–8.
- Pinto, V. G., Nesi, L. L., Miletto, M. C., and Schnorr, L. M. (2021). Providing in-depth performance analysis for heterogeneous task-based applications with starvz. *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 16–25.